

Classification

Rules & Tree-Based Methods

Data Mining
Howles

Rules

- Also known as Decision Rules
- “if then”
 - If tail AND barks then is a dog
- In many cases, decision rules can fit into a hierarchical structure to form a tree

Requirement:

Adequacy Condition

- Suppose we have a rule that states “if rainy and windy, do not play”
- The rule is valid iff we never play on a rainy and windy day
- No 2 instances can have all the same attribute values yet be assigned to different classes

Classification Using Trees

- We will use a tree to determine the classification of an attribute
- Using the training data, we'll construct a decision tree to tell us how to classify other unknown instances
- The tree orders the attributes to test in a hierarchical structure

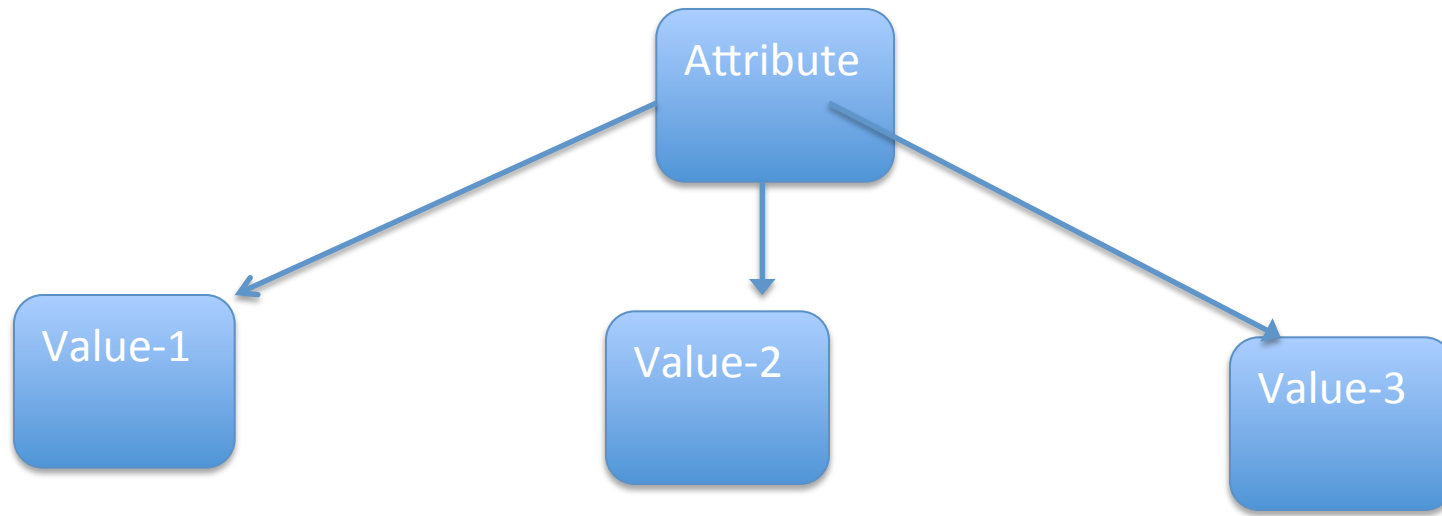
Internal Nodes & Leaves

- Internal nodes represent decisions
- Leaf nodes represent the classification
- Each node in the tree represents a decision or test. It may be a boolean (yes/no) or it may have several possible outcomes (small/medium/large)

Building the Tree

- A branch represents an outcome of a test. We continue following the branches until we reach a leaf node; the leaf represents the classification
- There are other issues such as missing values, or training data values that lead to different classifications -- we'll address some of these problems later

Sample Tree



An example

- The next slide shows the sample data set we will use
- It's the weather data we used before with continuous values for some attributes

Outlook	Temp	Humidity %	Windy?	Classification
Sunny	75	70	Yes	Play
Sunny	80	90	Yes	Don't Play
Sunny	85	85	No	Don't Play
Sunny	72	95	No	Don't Play
Sunny	69	70	No	Play
Overcast	72	90	Yes	Play
Overcast	83	78	No	Play
Overcast	64	65	Yes	Play
Overcast	81	75	No	Play
Rainy	71	80	Yes	Don't Play
Rainy	65	70	Yes	Don't Play
Rainy	75	80	No	Play
Rainy	68	80	No	Play
Rainy	70	96	No	Play

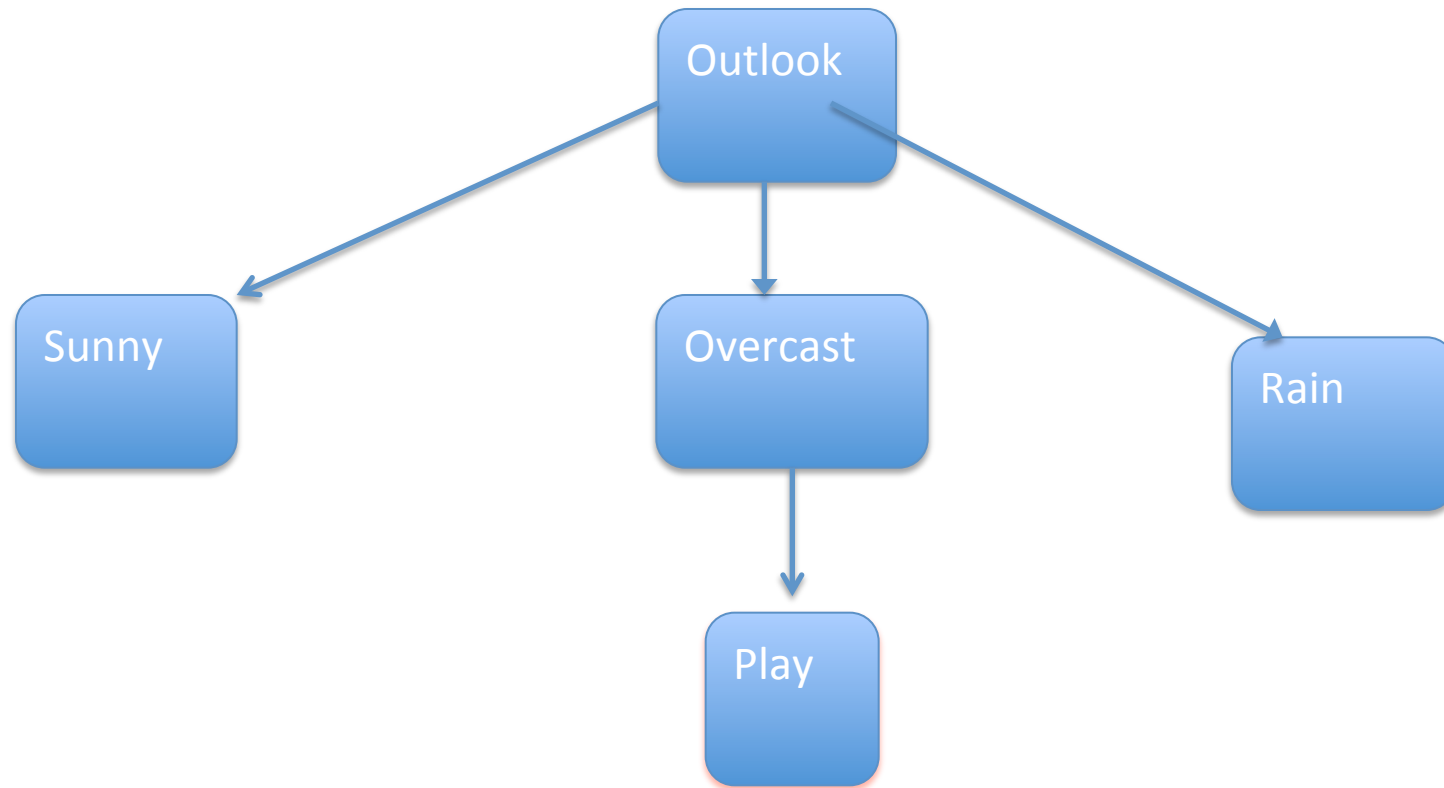
How to Split

- Take a good look at the data. Which attribute seems to be the best one to select for our root?
 - Remember we want to minimize the height of the tree, and be efficient by ruling out as much as possible early on in the classification process

What did you think?

- I hope you saw that "outlook" was the best candidate.
- Why? If the outlook is "overcast" -- we're done; no further decisions are necessary

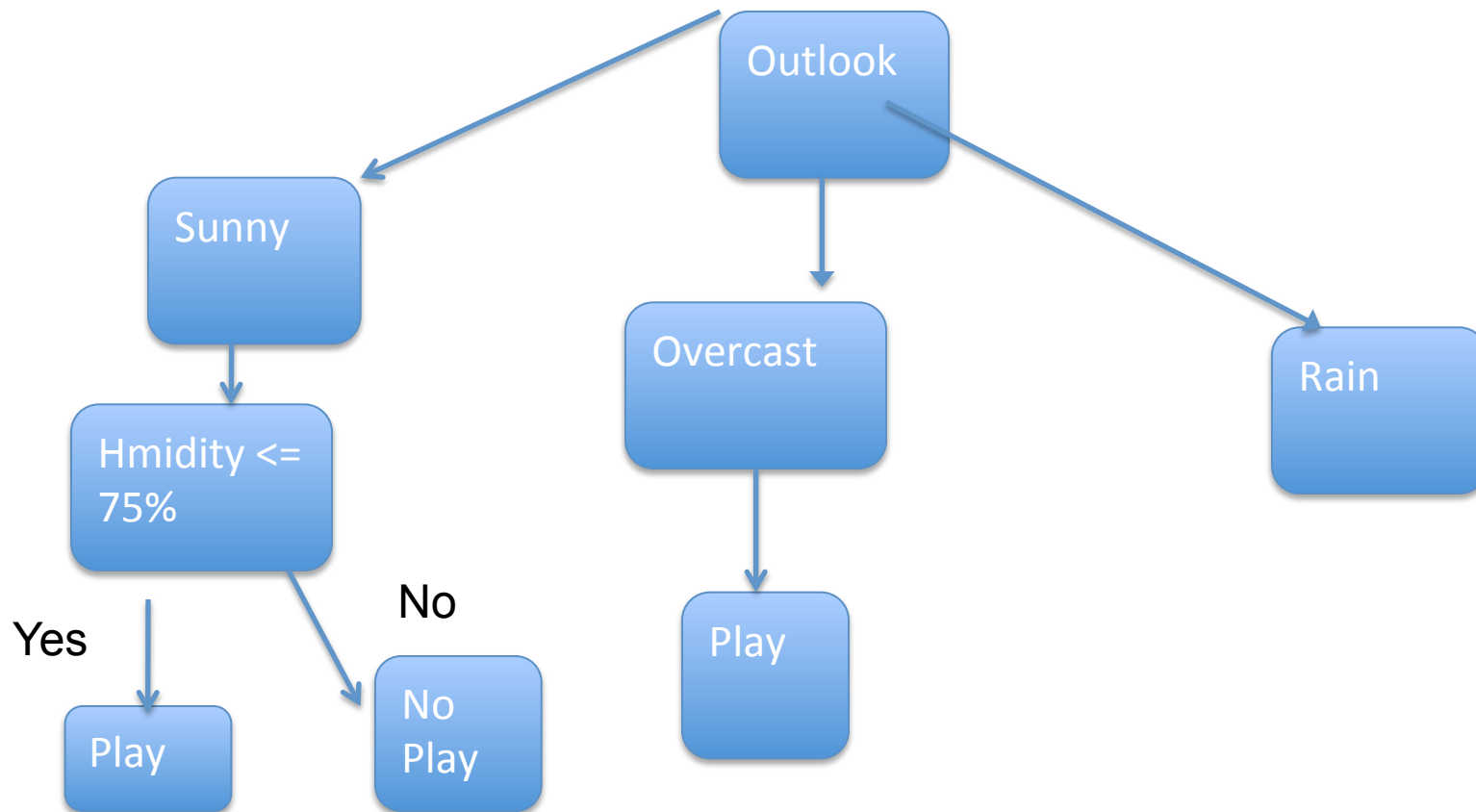
Initial Cut at the Tree



“Sunny” Subtree

- Look at the data again
- It is not decisive if it is sunny and windy; some play and others are no play
- What about the humidity? If $\leq 75\%$, play otherwise do not play
- Add the “Sunny” subtree

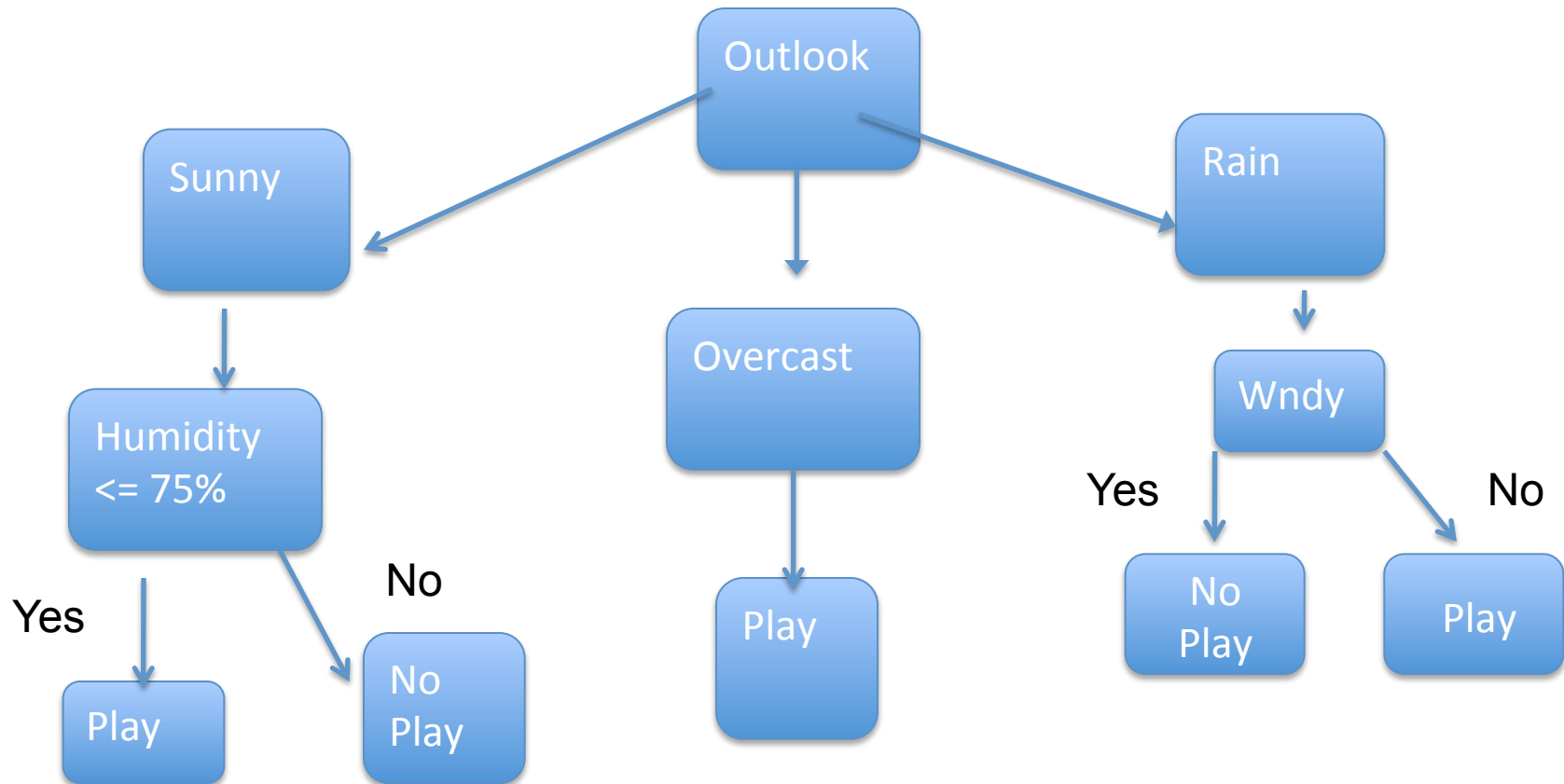
The Sunny Branch



What About the Rain Branch?

- Same idea, look at the Windy attribute
- Note we never used temperature

The Sunny Branch



Using the Tree

- You have a new instance
 - Sunny, 74 degrees, 77% humidity, no wind
 - Do we play?

Using the Tree

- You have a new instance
 - Sunny, 74 degrees, 77% humidity, no wind
 - Do we play?
- No, humidity is too high
- Note this “decision” is only a prediction based on previous events. It may be incorrect!

A More Complex Example

- The previous example was easy to build by evaluating each attribute then selecting the best one
- It was easy because we had a small data set with a limited number of attributes
- How do we build a tree if we have more data (a larger number of attributes)?

What Matters?

- Example: Student data includes high school GPA, major, average credit hours per term, hair color, age, if the student has a car on campus
- What is important in predicting student performance? Hair color and the car (probably) don't matter
- How do we determine which attributes are really important when making a classification?

Information Gain and Entropy

- What we need is a measurement to tell us which attributes give us the most information
- The information required to predict an event is the distribution's entropy
- Entropy gives the information required in bits (including partial bits)
- Entropy is the measure of the uncertainty in a training set because of possible multiple classifications
- Considering the information gain tends to find shorter trees

Deciding on the Best Split

- Purity - a measure telling us which nodes will produce the most efficient splits
- The measurement itself is represented in bits
- Entropy ($p_1, p_2, \dots, p_n$) =
 $-p_1 \log_2 p_1 - p_2 \log_2 p_2 \dots - p_n \log_2 p_n$

More About Trees

- Rules may be arbitrarily ordered and the tree will still be valid (height will be different)
- There may be attributes that do not impact the outcome
- Height impacts the classification time
- Maximum height is restricted by the number of attributes
- The algorithm is guaranteed to terminate

Missing Values

- How do we handle missing values?
- Several options – it's a decision based on what you are doing and the nature of the data
- In a tree, you can't really ignore or you won't have a path down the tree. So –
 - Create a separate branch
 - Estimate the value based on the other data

Reference Witten & Frank, *Data Mining*, Morgan Kaufmann, 2009

Overfitting

- Overfitting occurs when we rely too heavily on the training data
- We over-specify the rules
- As we saw before, overfitting can happen with other algorithms too

Addressing the problem

- Since overfitting is caused by writing rules too dependent on the training data, we can
 - Remove irrelevant attributes in the training data
 - Try to filter out noise (which may be contributing to incorrect classifications)
 - Attempt to limit the number of decisions – the number of branches
 - The more we branch, the more specific the classification becomes

Improving Results

- Studies have shown that additional, specific rules don't always produce better results
- A smaller number of more general rules tends to predict more accurate results when classifying previously unseen instances

Pruning

- To reduce the number of rules, prune the tree
- Either
 - Preprune (prune the data as you build the tree)
 - Postprune (build the tree, then prune)

Prepruning

- Use a terminating condition to know when to stop building new branches
- Typical methods include
 - terminating on the number of remaining instances
 - terminating when a certain number of branches is met

Terminating on the number of remaining instances

- When you get to the point where the number of instances involved is less than a determined threshold, stop
- Use the majority vote to determine the rule – we have previously done this with other algorithms
- This means you will be introducing errors, but it results in a less specific tree

Terminating - Instances

- Suppose we have this data:
 - 1, 2, 3, yes, yes, 8,... → Class C1
 - 1, 2, 3, yes, no, 8, ... → Class C2
 - 1, 2, 3, yes, no, 7,... → Class C1
- If these attributes are a, b, ..., our rule may be:
 - IF a = 1 AND b = 2 AND c = 3 AND d = yes Class is C1

Terminating - Instances

- We don't build a branch for the instance e,.. (assuming for this example, that by the time we get to this branch, the number of remaining values is small) OR
- That we are willing to use the most frequently occurring classification for the remaining instances (at the expense of introducing some classification errors)

Terminating – Branch Count

- Determine the maximum branch length and stop adding more branches when this number is met
- Again assumes that we are willing to use the most frequently occurring classification for the remaining instances (at the expense of introducing some classification errors)

Terminating – Branch Count

- It's difficult to determine the maximum branch length – depends on the data and the number of attributes (especially the *influential* attributes)
- What works for one data set may not work for another

Postpruning

- In practice, more common than using prepruning
- Build a complete tree
- Find all nodes which have only child leaf nodes
- Calculate the error rate if this node were pruned
- Prune if the error rate at this branch is greater than the estimated error rate - this branch is introducing more errors than estimated

Determining the Estimated Error Rates

- One way is to use a separate pruning set. Remove the branch, run the data and count the errors
- Another approach is to devise a formula to calculate the error rates. Would need to look specifically at the data, number of instances at different classifications, prior probabilities

Postpruning

- Extreme case, the tree is pruned back to the root
 - Hopefully rare
 - Essentially ZeroR rule
 - Remember from the lab – if your algorithm produces results no better than ZeroR, you may have something wrong with your algorithm or you are overfitting the training data

Other Tree Problems

- We said that the data must not violate the adequacy condition
- An instance with certain attribute values may not have a different classification from another instance with the exact same values
- But this does happen, often because of noisy data

Violation of the Adequacy Condition

Customer Code	Store Credit?	Tier	Class
1	Y	2	1
1	Y	2	2
1	Y	2	1

What do we do with this data?

We can build a leaf node “Clash Node” indicating we had more than one classification.

Or, we can throw out the branch, take the majority classification and recognize that we will have misclassifications.

Violation of the Adequacy Condition

- $X=1, Y=1, Z=1, \text{Class} = C1$
- $X=1, Y=1, Z=1, \text{Class} = C2$

We have a clash for this classification

